



What does this get us?



Theophrastus, and his imitators, are not the axiomatic

$$\begin{array}{c}
\frac{}{\text{when}(\bar{c})\{s_1\}; s_2 \hookrightarrow s_2 \mid (\bar{c}, s_1)} \text{STEP} \\
\\
\frac{s_1 \hookrightarrow s_2 \mid (\bar{c}_3, s_3) \quad j \text{ fresh}}{\langle \bar{b}_r[(i, \bar{c}_1, s_1)], \bar{b}_p \rangle / H \rightsquigarrow \langle \bar{b}_r[(i, \bar{c}_1, s_2)], \bar{b}_p : (j, \bar{c}_3, s_3) \rangle / H[i += S_j]} \text{SPAWN} \\
\\
\frac{C = \{c \mid c \in \bar{c}' \wedge (_, \bar{c}', _) \in \bar{b}_r \cup \bar{b}_{p1}\} \quad C \cap \bar{c} = \emptyset}{\langle \bar{b}_r, \bar{b}_{p1} : (i, \bar{c}, s) : \bar{b}_{p2} \rangle / H \rightsquigarrow \langle \bar{b}_r : (i, \bar{c}, s), \bar{b}_{p1} : \bar{b}_{p2} \rangle / H[i += R_i][\bar{c} += R_i]} \text{RUN} \\
\\
\frac{}{\langle \bar{b}_r[(i, \bar{c}, \text{done})], \bar{b}_p \rangle / H \rightsquigarrow \langle \bar{b}_r[\epsilon], \bar{b}_p \rangle / H[i += C_i][\bar{c} += C_i]} \text{COMPLETE}
\end{array}$$

■ **Figure 9** Semantics of the core BoC calculus with histories.

$$\begin{array}{c}
\frac{}{i \vdash_b \epsilon} \text{WFB-EMPTY} \quad \frac{}{i \vdash_b R_i : \bar{S} : (C_i?)} \text{WFB-NEMPTY} \\
\\
\frac{}{\vdash_c \epsilon} \text{WFC-EMPTY} \quad \frac{}{\vdash_c R_i} \text{WFC-SINGLE} \quad \frac{\vdash_c \bar{E}}{\vdash_c R_i : C_i : \bar{e}} \text{WFC-PAIR}
\end{array}$$

■ **Figure 10** Well-formedness rules for behaviour and cown histories

4.3 Soundness of Core Calculus According to the Axiomatic Model

In the previous section, we showed that a well-formed history translates to a valid execution in the axiomatic model. In this section we show that the semantics is sound according the axiomatic model by proving that it always produces a well-formed history.

In addition to general well-formedness, the history of an execution is going to match the specific configuration that it was produced together with.

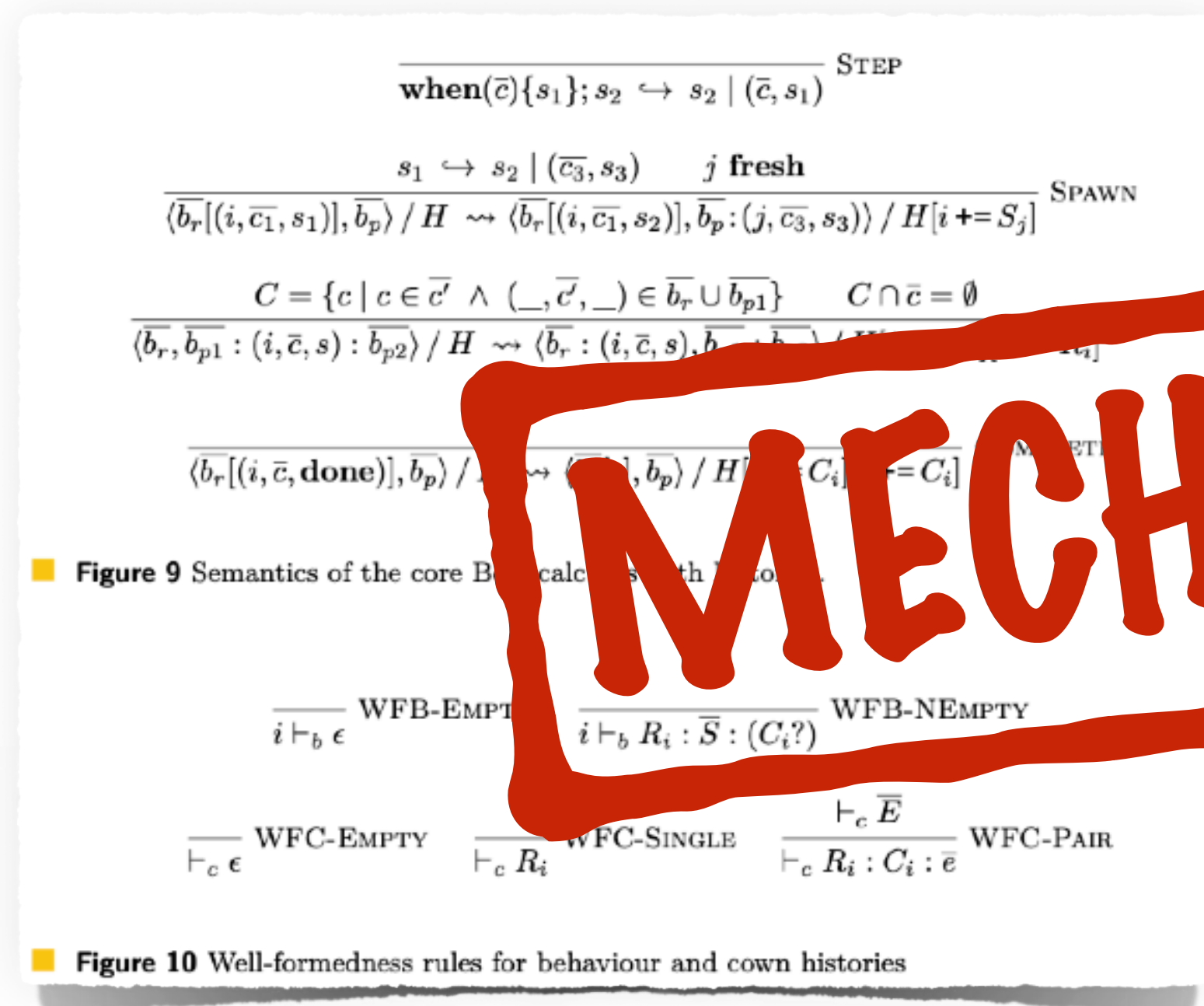
► **Theorem 15** (Preservation of well-formed and matching histories). *Starting from a well-formed and matching history, evaluation produces another well-formed and matching history:*

$$\tau \vdash H \wedge H; \tau \vdash c/g \wedge c/g/H \rightarrow c/g/H' \implies \exists \tau'. \tau' \vdash H' \wedge H'; \tau' \vdash c/g'$$

Proof. By cases on the evaluation relation. For each case we select a τ' that is τ extended so that the new event produced by that evaluation step is given a timestamp larger than any previous timestamp. Preservation of each of the two relations can then be proved separately. We refer to the mechanisation for details [7]. ◀

MECHANISED

What does this get us?



4.3 Soundness of Core Calculus According to the Axiomatic Model

In the previous section, we showed that the operational semantics translates to a valid execution in the axiomatic model. In this section we show that the axiomatic semantics is sound according to the operational model by proving that it also preserves well-formed history.

In addition to the well-formedness, the state of the execution is going to match the specific configuration that it is associated together with.

► **Theorem 15** (Preservation of well-formed and matching histories). *Starting from a well-formed and matching history, evaluation produces a well-formed and matching history:*

$$i \vdash_b \epsilon \quad \langle \bar{b}_r, \bar{b}_p \rangle / H \rightarrow \langle \bar{b}_r', \bar{b}_p' \rangle / H' \implies \exists \tau'. \tau' \vdash H' \wedge H'; \tau' \vdash \langle \bar{b}_r', \bar{b}_p' \rangle$$

Proof. By cases on the evaluation relation. For each case we select a τ' that is τ extended so that the new event produced by that evaluation step is given a timestamp larger than any previous timestamp. Preservation of each of the two relations can then be proved separately. We refer to the mechanisation for details [7]. ◀

The operational semantics, and its implementations, are sound with respect to the axiomatic model

What does this get us?

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

	src=0, dst=0	src=0, dst=10	src=10, dst=0	src=10, dst=10
Order	C B D A C D A B C D B A D A C B D C A B D C B A	B C D A B D A C B D C A D A B C D A B C D B C A	A C B D A C D B A D C B C A B D C A D B C B A D	A B C D A B D C A D B C B A C D B A D C B C A D
Permitted	✓	⚠	✓	✓